REST API

Documentation

Version 1.3 | April 2026

Base URL: http://localhost/api

Overview

This document describes all available REST API endpoints. Unless otherwise noted, all POST endpoints require the user to be authenticated via a session cookie. GET endpoints are generally public.

Authentication

Session-based authentication is used. After logging in, a session cookie (cookies.txt) must be sent with all subsequent authenticated requests.

Permission Scopes

Many resources are protected by scope values attached to a user role:

Value	Meaning
0	`NONE — No access`
1	`SELF — Access limited to own resources`
2	`ALL — Full access`

Authentication

No authentication required for these endpoints.

Method	Endpoint	Description
POST	`/auth/register`	Create a new user account (role set to PUBLIC by default)
POST	`/auth/login`	Log in and receive a session cookie
GET	`/auth/me`	Check current session / logged-in user
POST	`/auth/logout`	Destroy the current session

POST /auth/register

Creates a user account. The user role is set to PUBLIC by default.

Example Request:

```
curl -i -b cookies.txt \
  -H "Content-Type: application/json" \
  -X POST http://localhost/api/auth/register \
  -d '{
    "personId": "test_1",
    "firstName": "Test",
    "lastName": "User",
    "email": "user@example.com",
    "addressLineOne": "123 Test St",
    "addressLineTwo": "Apt 4",
    "city": "Fayetteville",
    "stateProvince": "AR",
    "zipCode": "72701",
    "country": "US",
    "primaryPhone": "4795551234",
    "secondaryPhone": null,
    "notes": "Created via auth register endpoint",
    "password": "dog12345"
  }'
```

POST /auth/login

Logs in a user. On success, a session cookie is returned and should be stored for subsequent requests.

Example Request:

```
curl -i -c cookies.txt -b cookies.txt \
  -H "Content-Type: application/json" \
  -X POST http://localhost/api/auth/login \
  -d '{"username": "test_1", "password": "dog12345"}'
```

GET /auth/me

Returns the current session user's details. Useful for debugging and session validation.

Example Request:

```
curl -i -b cookies.txt http://localhost/api/auth/me
```

POST /auth/logout

Destroys the current session and logs out the user.

Example Request:

```
curl -i -c cookies.txt -b cookies.txt -X POST http://localhost/api/auth/logout
```

Person

Manage person/user accounts.

Permissions — Only Admins allowed

Method	Endpoint	Description
POST	`/person/add`	Create a new person/account
POST	`/person/edit`	Edit a person's details
POST	`/person/change-password`	Change your own password
POST	`/person/change-user-role`	Change a person's system role
GET	`/person/get`	List all people
GET	`/person/get/{person_id}`	Get a person by ID
GET	`/person/mine`	Get own account information
GET	`/person/search?q=`	Search by first name, last name, email, or role
GET	`/person/is-secretary/{person_id}`	Check if a person is a secretary
GET	`/person/is-board-member/{person_id}`	Check if a person is a board member
GET	`/person/is-judge/{person_id}`	Check if a person is a judge

POST /person/add — Example Request:

```
curl -i -b cookies.txt \
  -H "Content-Type: application/json" \
  -X POST http://localhost/api/person/add \
  -d '{
    "personId": "jdoe2",
    "firstName": "John",
    "lastName": "Doe",
    "email": "jdoe@test.com",
    "password": "password123"
  }'
```

POST /person/change-user-role — Example Request:

```
curl -i -b cookies.txt \
  -H "Content-Type: application/json" \
  -X POST http://localhost/api/person/change-user-role \
  -d '{"personId": "test_1", "systemRole": "ADMIN"}'
```

Dog

Manage dog records. Adding or editing a dog will automatically assign titles based on current stats.

Permissions — EditDogScope

Level	Scope	Permissions
0	NONE	Cannot create, edit, or delete dogs.
1	SELF	Can create dogs (unrestricted, since ownership is assigned later). Can edit/delete only dogs you own via DogOwner.
2	ALL	Can create, edit, or delete any dog.

Method	Endpoint	Description
POST	`/dog/add`	Add a dog (auto-assigns titles based on stats)
POST	`/dog/edit`	Edit a dog (auto-assigns titles based on stats)
POST	`/dog/delete`	Delete a dog
GET	`/dog/get`	List all dogs
GET	`/dog/get/{cwa_number}`	Get a specific dog
GET	`/dog/my-dogs`	Get all dogs owned by the current user
GET	`/dog/titles/{cwa_number}`	Calculate eligible titles based on stats
GET	`/dog/grade/{cwa_number}`	Compute the grade of a dog
GET	`/dog/age/{cwa_number}`	Calculate dog age
GET	`/dog/meets/{cwa_number}`	Get meets, meet results, and race results for a dog
GET	`/dog/wins/{cwa_number}`	Get DPC wins and meet wins for a dog
GET	`/dog/arx/{cwa_number}`	Check ARX title eligibility
GET	`/dog/narx/{cwa_number}`	Check NARX title eligibility
GET	`/dog/trp/{cwa_number}`	Check TRP title eligibility
GET	`/dog/pr/{cwa_number}`	Check PR title eligibility
GET	`/dog/dpc/{cwa_number}`	Check DPC title eligibility
GET	`/dog/hc/{cwa_number}`	Check HC title eligibility
GET	`/dog/search?q=`	Search by name, owner, call name, or CWA number

POST /dog/add — Example Request:

```
curl -i -b cookies.txt \
  -H "Content-Type: application/json" \
  -X POST http://localhost/api/dog/add \
  -d '{
    "cwaNumber": "T12345",
    "registeredName": "TEST DOG",
    "status": "Active",
    "currentGrade": "A",
    "callName": "TEST",
    "birthdate": "2020-01-01",
    "akcNumber": null,
    "meetPoints": "0",
    "arxPoints": "0",
    "narxPoints": "0",
    "showPoints": "0",
    "dpcLegs": "0",
    "meetWins": "0",
    "meetAppearences": "0",
    "highCombinedWins": "0",
    "notes": "Created via curl"
  }'
```

Dog Owner

Manage the ownership link between a person and a dog.

Permissions — EditDogScope

Method	Endpoint	Description
POST	`/dog_owner/add`	Link a person to a dog as owner
POST	`/dog_owner/delete`	Remove a dog-owner link
POST	`/dog_owner/transfer`	Transfer ownership to a new person
GET	`/dog_owner/get`	List all dog-owner records
GET	`/dog_owner/owners/{cwa_number}`	Get all owners for a specific dog

POST /dog_owner/add — Example Request:

```
curl -i -b cookies.txt \
  -H "Content-Type: application/json" \
  -X POST http://localhost/api/dog_owner/add \
  -d '{"cwaId": "T12345", "personId": "test_1"}'
```

POST /dog_owner/transfer — Example Request:

```
curl -i -b cookies.txt \
  -H "Content-Type: application/json" \
  -X POST http://localhost/api/dog_owner/transfer \
  -d '{"cwaId": "T12345", "newOwnerPersonId": "test_1"}'
```

Dog Title

Manage titles awarded to dogs.

Permissions — EditDogScope

Method	Endpoint	Description
POST	`/dog_title/add`	Add a title to a dog
POST	`/dog_title/edit`	Edit a dog's title
POST	`/dog_title/delete`	Delete a dog's title
GET	`/dog_title/get`	List all dog titles
GET	`/dog_title/get/{cwa_number}/{title}`	Get a specific title for a dog

POST /dog_title/add — Example Request:

```
curl -i -b cookies.txt \
  -H "Content-Type: application/json" \
  -X POST http://localhost/api/dog_title/add \
  -d '{
    "cwaNumber": "T12345",
    "title": "ARX",
    "titleDate": "2024-05-01",
    "titleNumber": "1",
    "namePrefix": "A",
    "nameSuffix": "X"
  }'
```

Meet

Manage racing meets. GET endpoints are public; POST endpoints require authentication.

Permissions — EditMeetScope

Level	Scope	Permissions
0	NONE	Cannot create, edit, or delete meets.
1	SELF	Can create, edit, or delete meets only if you are listed as judge or raceSecretary on the meet.
2	ALL	Can create, edit, or delete any meet.

Method	Endpoint	Description
POST	`/meet/add`	Add a meet
POST	`/meet/edit`	Edit a meet
POST	`/meet/delete`	Delete a meet
GET	`/meet/get`	List all meets
GET	`/meet/get/{meet_number}`	Get a specific meet

POST /meet/add — Example Request:

```
curl -i -b cookies.txt \
  -H "Content-Type: application/json" \
  -X POST http://localhost/api/meet/add \
  -d '{
    "meetNumber": "MEET-001",
    "clubAbbreviation": "TCLUB",
    "meetDate": "2026-04-10",
    "raceSecretary": "SYSTEM",
    "judge": "SYSTEM",
    "location": "LITTLE ROCK",
    "yards": "200"
  }'
```

Meet Result

Manage results for a dog in a given meet. Adding or editing a result updates the dog's stats and auto-assigns titles.

Permissions — EditMeetScope

Method	Endpoint	Description
POST	`/meet_result/add`	Add a meet result (updates dog stats & auto-assigns titles)
POST	`/meet_result/edit`	Edit a meet result (updates dog stats & auto-assigns titles)
POST	`/meet_result/delete`	Delete a meet result
GET	`/meet_result/get`	List all meet results
GET	`/meet_result/get/{meet_number}/{cwa_number}`	Get the result for a specific dog in a meet

POST /meet_result/add — Example Request:

```
curl -i -b cookies.txt \
  -H "Content-Type: application/json" \
  -X POST http://localhost/api/meet_result/add \
  -d '{
    "meetNumber": "MEET-001",
    "cwaNumber": "T12345",
    "average": "9.2",
    "grade": "A",
    "meetPlacement": "3",
    "meetPoints": "1",
    "arxEarned": "2",
    "narxEarned": "4",
    "shown": "0",
    "showPlacement": "1",
    "showPoints": "3",
    "dpcLeg": "2",
    "hcScore": "2",
    "hcLegEarned": "2"
  }'
```

Race Result

Manage individual race results within a meet.

Permissions — EditMeetScope

Method	Endpoint	Description
POST	`/race_result/add`	Add a race result. (updates meet results, dog stats & auto-assigns titles)
POST	`/race_result/edit`	Edit a race result (updates meet results, dog stats & auto-assigns titles)
POST	`/race_result/delete`	Delete a race result
GET	`/race_result/get`	List all race results
GET	`/race_result/get/{meet}/{cwa}/{program}/{race}`	Get a specific race result

POST /race_result/add — Example Request:

```
curl -i -b cookies.txt \
  -H "Content-Type: application/json" \
  -X POST http://localhost/api/race_result/add \
  -d '{
    "meetNumber": "MEET-001",
    "cwaNumber": "T12345",
    "program": "A",
    "raceNumber": "1",
    "entryType": "REG",
    "box": "3",
    "placement": "1",
    "meetPoints": "4",
    "incident": ""
  }'
```

Stats

Public statistics endpoints. No authentication or permissions required.

Method	Endpoint	Description
GET	`/stats/top/all-time/{limit}`	Top N dogs of all time (sorted by points then wins)
GET	`/stats/top/{year}/{limit}`	Top N dogs for a specific year
GET	`/stats/search/year/{year}`	Search stats by year; optionally filter by dog and owner
GET	`/stats/search/dog/{dog_id}`	Search stats by dog; optionally filter by year and owner
GET	`/stats/search/owner/{owner_id}`	Search stats by owner; optionally filter by dog and year
GET	`/stats/years`	List all years for which stats data exists

GET /stats/top/all-time/{limit} — Example Request:

```
curl -i -b cookies.txt "http://localhost/api/stats/top/all-time/20"
```

GET /stats/search/dog/{dog_id} with filters — Example Request:

```
curl -i -b cookies.txt
"http://localhost/api/stats/search/dog/T12345?year=2026&owner_id=test_1"
```

User Role

Manage user role definitions and their scope configurations. ADMIN and PUBLIC are protected roles and cannot be edited or deleted.

Permissions — Only Admins allowed

Method	Endpoint	Description
POST	`/user_role/add`	Add a user role
POST	`/user_role/edit`	Edit a user role
POST	`/user_role/delete`	Delete a user role
GET	`/user_role/get`	List all user roles
GET	`/user_role/get/{role_id}`	Get a specific user role

POST /user_role/add — Example Request:

```
curl -i -b cookies.txt \
  -H "Content-Type: application/json" \
  -X POST http://localhost/api/user_role/add \
  -d '{
    "title": "SECRETARY",
    "editDogScope": 1,
    "editPersonScope": 1,
    "editDogOwnerScope": 1,
    "editOfficerRoleScope": 0,
    "editUserRoleScope": 0,
    "editClubScope": 1,
    "editMeetScope": 2,
    "editMeetResultsScope": 2,
    "editRaceResultsScope": 2,
    "editDogTitlesScope": 1,
    "editTitleTypeScope": 0,
    "editNewsScope": 1
  }'
```

Title Type

Manage the canonical list of title types and their descriptions.

Permissions — EditTitleTypeScope

Level	Scope	Permissions
0	NONE	Cannot create, edit, or delete title types.
1	SELF	Can create, edit, or delete title types.
2	ALL	Can create, edit, or delete title types.

Method	Endpoint	Description
POST	`/title_type/add`	Add a title type
POST	`/title_type/edit`	Edit a title type
POST	`/title_type/delete`	Delete a title type
GET	`/title_type/get`	List all title types
GET	`/title_type/get/{title}`	Get a title type by name

POST /title_type/add — Example Request:

```
curl -i -b cookies.txt \
  -H "Content-Type: application/json" \
  -X POST http://localhost/api/title_type/add \
  -d '{"title": "ARX", "titleDescription": "Award of Racing Excellence"}'
```

Importer

Bulk import dogs, meets, meet results, and race results via CSV file upload.

Permissions — Only Admins allowed

Method	Endpoint	Description
POST	`/import?type=dogs&mode=insert`	Import dogs from CSV
POST	`/import?type=meets&mode=insert`	Import meets from CSV (club must exist)
POST	`/import?type=meet_results&mode=insert`	Import meet results from CSV (meet and dogs must exist)
POST	`/import?type=race_results&mode=insert`	Import race results from CSV (dogs and meet results must exist)
GET	`/import/types`	List all available import types

POST /import?type=dogs&mode=insert — Example Request:

```
curl -i -b cookies.txt \
  -X POST "http://localhost/api/import?type=dogs&mode=insert" \
  -F "file=@backend/test/dog_import.csv"
```

Change Log

View change log entries. No authentication or permissions required.

Method	Endpoint	Description
GET	`/change_log/get`	Get all change log entries
GET	`/change_log/get/{id}`	Get a specific change log entry by ID

GET /change_log/get/ — Example Request:

```
curl -i -b cookies.txt http://localhost/api/change_log/get
```

GET /change_log/get/{id} — Example Request:

```
curl -i -b cookies.txt http://localhost/api/change_log/get/1
```

Database

Dump and restore the database.

Permissions — Only Admins allowed

Method	Endpoint	Description
GET	`/database/dump`	Download the current database as a SQL dump
POST	`/database/restore`	Restore a database from an uploaded SQL dump file

GET /database/dump — Example Request:

```
curl -b cookies.txt -X GET http://localhost/api/database/dump -o db_dump.sql
```

POST /database/restore — Example Request:

```
curl -i -b cookies.txt -X POST http://localhost/api/database/restore -F "file=@db_dump.sql"
```